

Blockchain Experiment 6

Name: Atharva Lotankar

Class: D20C **R.No.:** 20

Date: 23 / 09 / 2026

Aim: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.

Theory:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce **security, correctness, and reliability** in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

• *mapping*

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is mapping(keyType => valueType). For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

- **storage**

In Solidity, storage refers to the **permanent memory of the contract**, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

- **memory**

In contrast, memory is **temporary storage**, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

3. Why bytes32 Instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies in *efficiency and gas optimization*.

- **bytes32** is a **fixed-size type**, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a **dynamically sized type**, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

4. Proportional Budget Allocation Math

Instead of selecting a single winner, this contract divides a treasury pool among clubs proportionally based on their accumulated votes:

$$\text{Allocated Budget} = (\text{Club Votes} * \text{Total Treasury Pool}) / (\text{Total Votes Cast})$$

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the **Web3 Type Converter** help developers easily switch between these two types for deployment and testing.

In summary, bytes32 is used when performance and gas efficiency are priorities, while string is preferred for readability and ease of use.

Program Code:

CampusBudgetAllocator.sol

```
// SPDX-License-Identifier: MIT
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title CampusBudgetAllocator
 * @dev Manages campus club budget
allocation using weighted department
voting.
 */
contract CampusBudgetAllocator {

    struct Representative {
        uint studentWeight; // Number
of students represented
        bool hasVoted;      // Has the
representative voted?
        address delegate;   // Person
delegated to
        uint votedClubIndex; // Index
of the chosen club
    }

    struct Club {
        string name;        // Name of
the campus club
        uint totalVotes;    //
Accumulated votes from departments
        uint allocatedBudget; //
Calculated funding amount
    }

    address public dean;
    uint public totalTreasuryPool; //
Total budget in Wei or arbitrary units
    uint public totalVotesCast;

    mapping(address => Representative)
public representatives;
    Club[] public clubs;

    constructor(string[] memory
clubNames, uint _totalTreasuryPool) {
        dean = msg.sender;
        totalTreasuryPool =
_totalTreasuryPool;
```

```
        for (uint i = 0; i <
clubNames.length; i++) {
            clubs.push(Club({
                name: clubNames[i],
                totalVotes: 0,
                allocatedBudget: 0
            }));
        }

        // Register a representative with
dynamic department weight
        function
registerRepresentative(address
repAddress, uint departmentSize)
external {
            require(msg.sender == dean,
"Only the Dean can register reps.");

            require(!representatives[repAddress].h
asVoted, "Rep already voted.");

            require(representatives[repAddress].st
udentWeight == 0, "Rep already
registered.");
            require(departmentSize > 0,
"Department size must be greater than
zero.");

            representatives[repAddress].studentWei
ght = departmentSize;
        }

        // Delegate voting weight to
another representative
        function delegateVote(address to)
external {
            Representative storage sender
= representatives[msg.sender];
            require(sender.studentWeight >
0, "You have no voting rights.");
            require(!sender.hasVoted, "You
already voted.");
            require(to != msg.sender,
"Self-delegation not allowed.");
```

```

        while
        (representatives[to].delegate !=
address(0)) {
            to =
representatives[to].delegate;
            require(to != msg.sender,
"Loop detected in delegation.");
        }

        Representative storage
targetDelegate = representatives[to];

        require(targetDelegate.studentWeight
>= 1, "Target cannot receive votes.");

        sender.hasVoted = true;
        sender.delegate = to;

        if (targetDelegate.hasVoted) {

clubs[targetDelegate.votedClubIndex].t
otalVotes += sender.studentWeight;
            totalVotesCast +=
sender.studentWeight;
        } else {

targetDelegate.studentWeight +=
sender.studentWeight;
        }
    }

    // Cast vote for a club
    function voteForClub(uint
clubIndex) external {
        Representative storage sender
= representatives[msg.sender];
        require(sender.studentWeight >
0, "Not authorized to vote.");
        require(!sender.hasVoted,
"Already voted.");
        require(clubIndex <
clubs.length, "Invalid club choice.");

```

```

        sender.hasVoted = true;
        sender.votedClubIndex =
clubIndex;

        clubs[clubIndex].totalVotes +=
sender.studentWeight;
        totalVotesCast +=
sender.studentWeight;
    }

    // Calculate budget split based on
percentage share of total votes
    function
calculateBudgetDistribution() external
{
        require(msg.sender == dean,
"Only the Dean can execute payout
calculations.");
        require(totalVotesCast > 0,
"No votes were cast.");

        for (uint i = 0; i <
clubs.length; i++) {
            clubs[i].allocatedBudget =
(clubs[i].totalVotes *
totalTreasuryPool) / totalVotesCast;
        }
    }

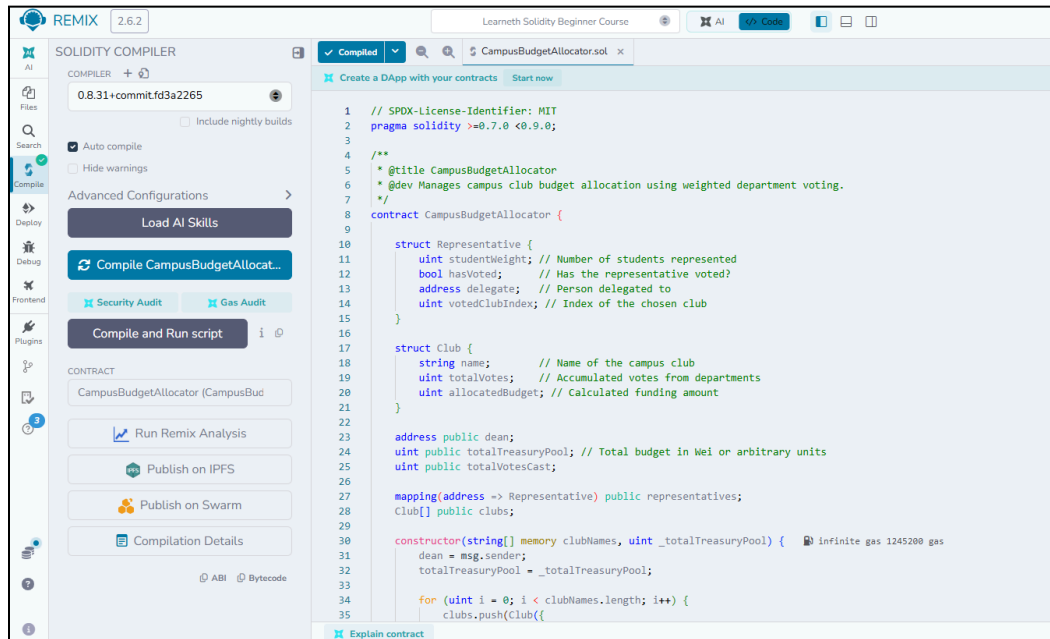
    // Fetch allocated budget for a
specific club index
    function getClubDetails(uint
clubIndex) external view returns
(string memory name, uint totalVotes,
uint allocatedBudget) {
        require(clubIndex <
clubs.length, "Invalid club index.");
        Club memory c =
clubs[clubIndex];
        return (c.name, c.totalVotes,
c.allocatedBudget);
    }
}

```

Implementation:

Step 1: Compilation of Smart Contract

The CampusBudgetAllocator.sol contract was compiled using Solidity Compiler v0.8.31 without any errors.

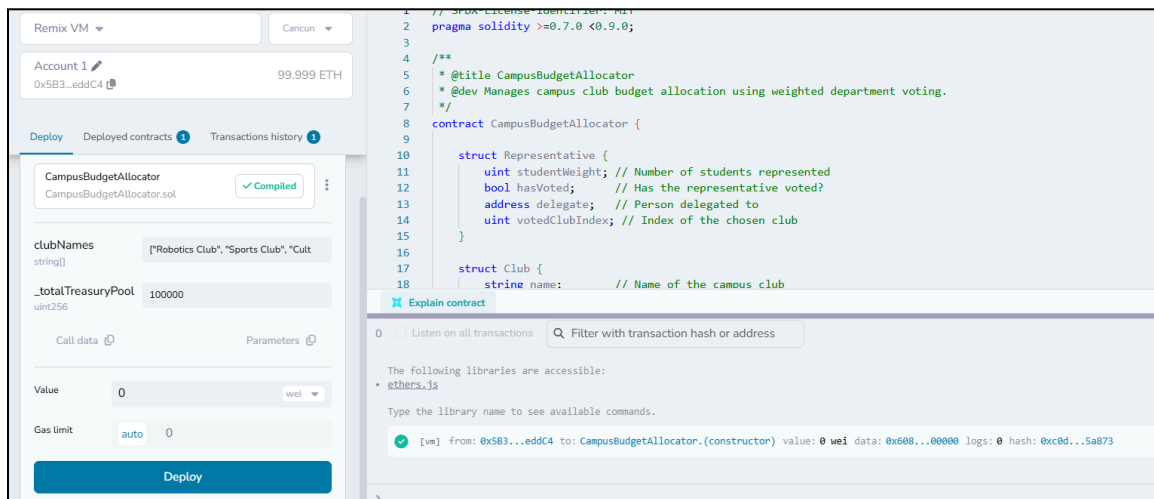


Step 2: Deployment of Smart Contract

The contract was deployed by the Dean (Account 1: 0x5B3...eddC4)

→ Constructor Arguments:

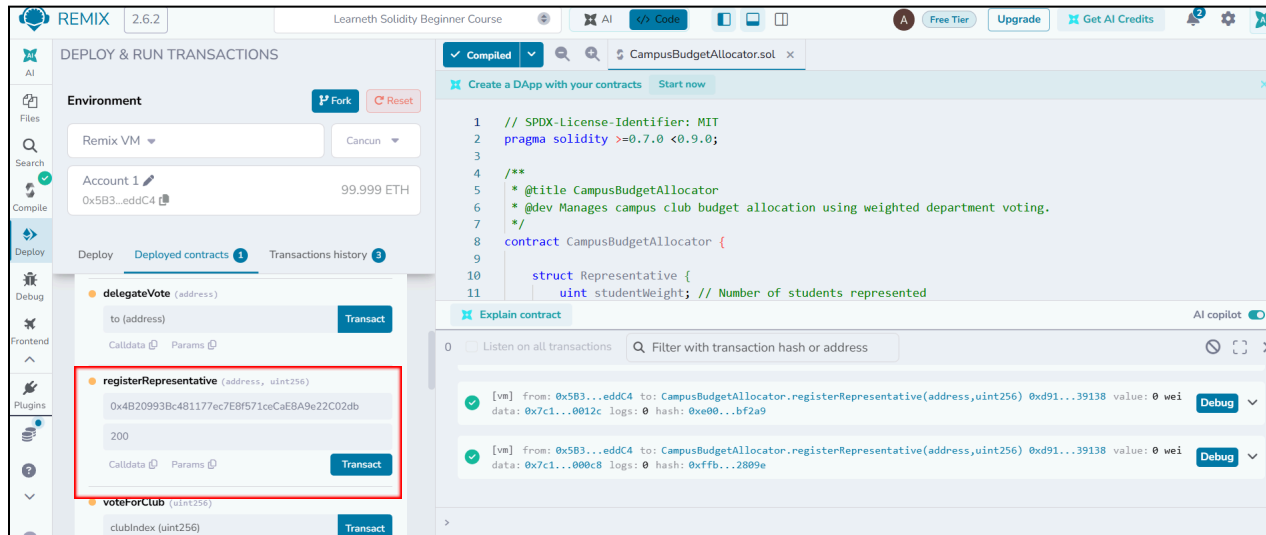
- clubNames: ["Robotics Club", "Sports Club", "Cultural Fest"]
- _totalTreasuryPool: 100000



Step 3: Registration of Department Representatives

The Dean registered two department representatives using **registerRepresentative**:

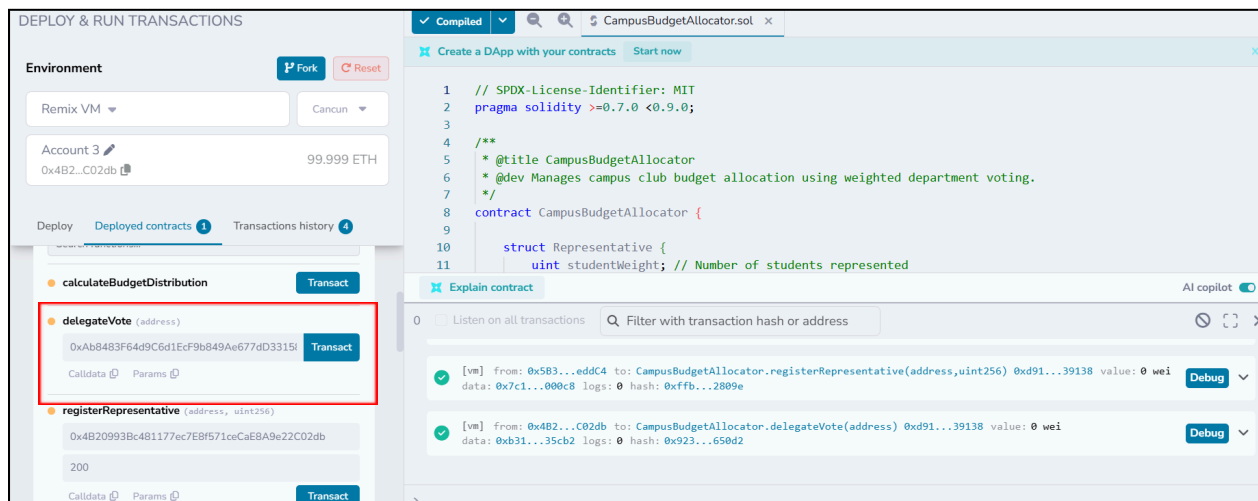
1. Representative 1 (Account 2: 0xAb8...35cb2): Assigned weight of 300 students.
2. Representative 2 (Account 3: 0x4B2...C02db): Assigned weight of 200 students.



Step 4: Vote Delegation

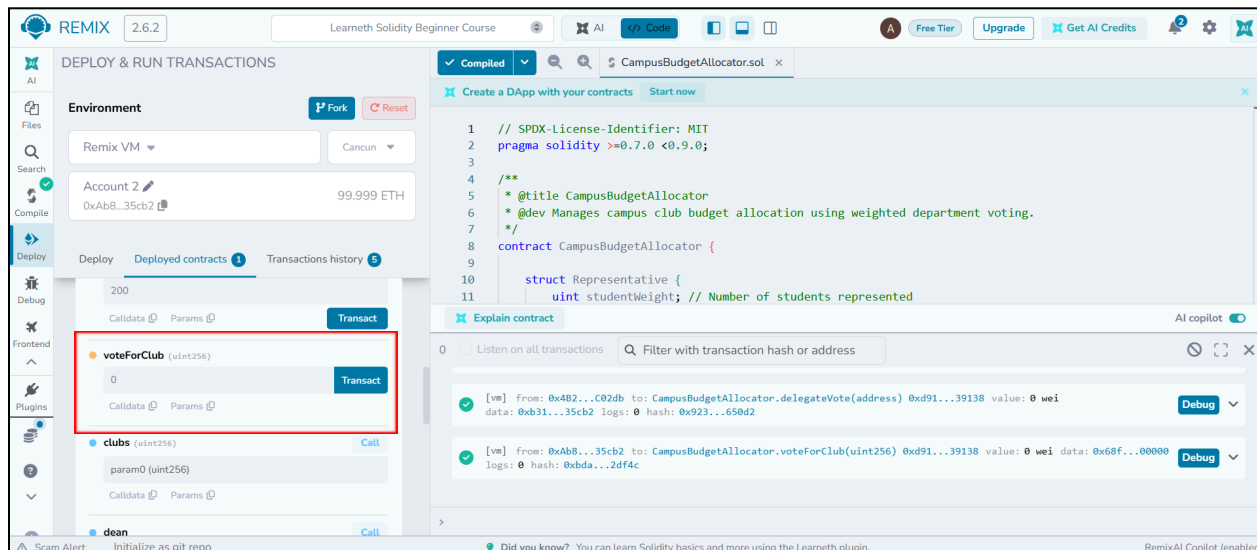
Representative 2 (0x4B2...C02db) called **delegateVote** to delegate their department's 200 votes to Representative 1 (0xAb8...35cb2).

- Result: Representative 1's combined weight became $300 + 200 = 500$ votes



Step 5: Casting Vote

Representative 1 (0xAB8...35cb2) executed `voteForClub(0)` to cast all 500 votes for **Robotics Club** (Index 0)

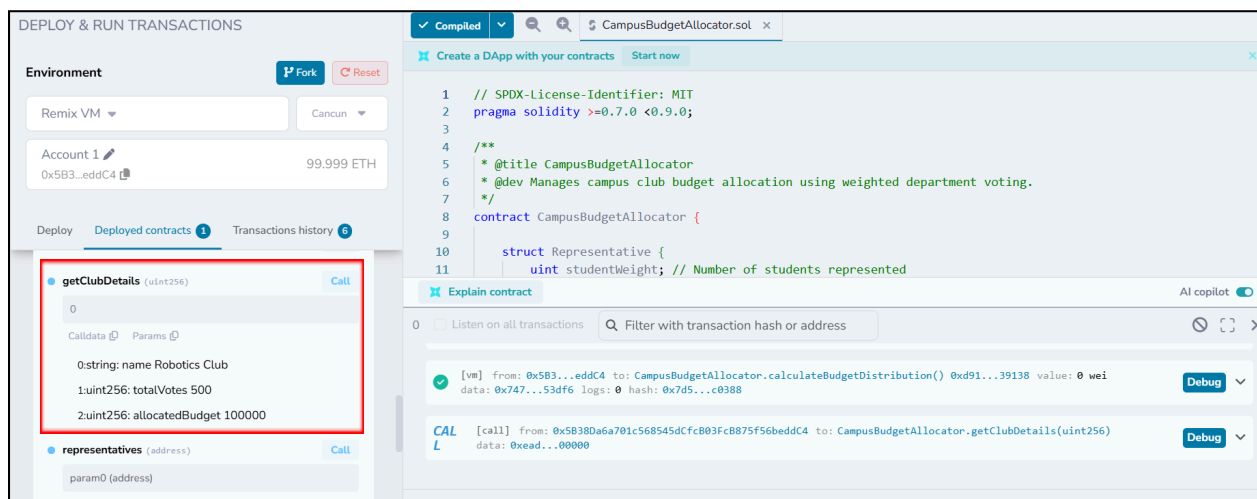


Step 6: Budget Calculation and Final Tally Verification

The Dean executed `calculateBudgetDistribution()`.

Two separate state queries were made using `getClubDetails` to verify the math[cite: 5, 11]:

1. Winning Club Query — `getClubDetails(0)` [Robotics Club]:
 - 0: string: name: Robotics Club
 - 1: uint256: totalVotes: 500
 - 2: uint256: allocatedBudget: 100000 (Received 100% of the treasury pool)



2. Non-Winning Club Query — `getClubDetails(1)` [Sports Club]:
 - 0: string: name: Sports Club

- 1: uint256: totalVotes: 0
- 2: uint256: allocatedBudget: 0 (Received 0% of the treasury pool)

The screenshot shows the Remix IDE interface. On the left, the 'Environment' panel shows the account 'Account 1' with 99.999 ETH. The 'Deployed contracts' list shows 'getClubDetails (uint256)' with a 'Call' button. The main editor displays the Solidity code for the 'CampusBudgetAllocator' contract. The bottom panel shows the transaction history with two transactions: a successful deployment and a successful call to 'getClubDetails'.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity >=0.7.0 <0.9.0;
3
4 /**
5  * @title CampusBudgetAllocator
6  * @dev Manages campus club budget allocation using weighted department voting.
7  */
8 contract CampusBudgetAllocator {
9
10     struct Representative {
11         uint studentWeight; // Number of students represented

```

Transaction history:

- [vm] from: 0x5B3...eddC4 to: CampusBudgetAllocator.calculateBudgetDistribution() @xd91...39138 value: 0 wei data: 0x747...53df6 logs: 0 hash: 0xc76...f1c01
- [call] from: 0x5B380a6a701c56854dCfc803Fc8875f56beddC4 to: CampusBudgetAllocator.getClubDetails(uint256) data: 0xead...00001

Conclusion:

In this experiment, a custom **Campus Club Budget Allocator** smart contract was successfully written, compiled, and deployed using Remix IDE. The deployment successfully demonstrated dynamic department weighting, vote delegation, and automated treasury distribution math on the blockchain. Important Ethereum concepts such as **require** access control, **storage** vs. **memory** data locations, and state variable updates were practically verified through the execution steps.